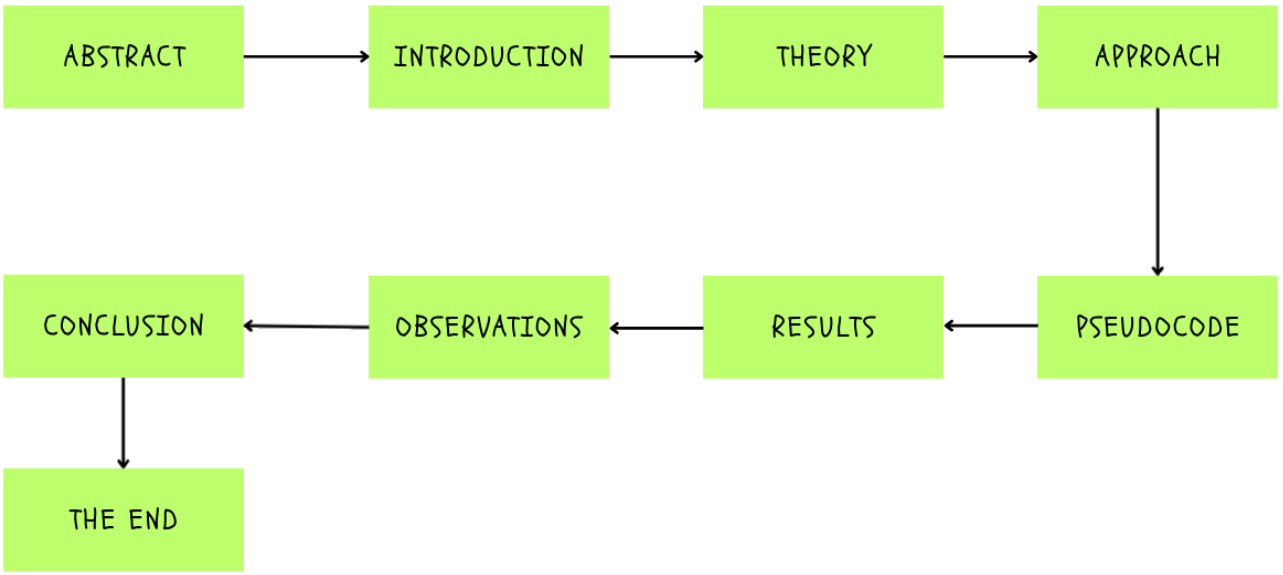
GESTURE VOLUME CONTROL

Report by:
Akhilesh (241EC106)
Manikanta (241EC132)
National Institute Of Technology Karnataka, Surathkal

Flow-

We will be dividing this whole report into parts and getting into details of it accordingly. The flow-chart of the whole report is as shown in the following diagram.



Abstract-

Technology is evolving at a very rapid speed. Automating systems is very essential for convenience and speed. Speaking of automation, we cannot completely automate certain technologies but we can create more convenient paths to interact with technologies we are using. For example, we cannot completely automate a system to directly understand what a person who cannot speak is telling but we can create a technology to understand one's hand gestures and use it to fill the gap of communication barriers. We can use gesture control systems to control certain things like volume, brightness, or on an industrial level, some of the machines which could be difficult to control otherwise. Gesture control systems are bringing revolution in the tech we are using in daily life. Therefore, this article deals with gesture control. To keep it to a minimal level we are going to implement a gesture control system to control volume. We will be using numpy, openCV, and other libraries and packages of python.

Introduction-

The gesture control system that we are building requires three basic functions to work. One, the ANN model has to be trained with the data of hands so that it can recognise what kind of gestures are being made and provide the result accordingly. Second, use the segmentation process to identify the connected regions in an image and lastly, create a system using python where you can use the data provided by the above functions and normalise the data using a fast fourier algorithm to get utmost accuracy. There are various other approaches to build a gesture-recognised volume controller, some of them use depth-sensing cameras which are expensive, some of them use change-in-colour recognition of your gesture to implement the above. But, we are going to stick to the simple yet a very powerful and high accuracy method.

Theory-

Human-Computer Interaction (HCI) is a powerful subject of study. It aims at replacing knobs, controllers,etc. With the normal kind of communication which can ease the communication barrier as well as speed up the technology at a faster rate.
Speaking of our project, the very first thing we need to do is **Spatial Acquisition:** Converting physical hand positions in 3D space into 2D coordinates (x,y) on a video frame.
The next step includes identifying specific positions or landmarks from the video frame and calculating geometric distance and other measurements between those landmarks. This process is called **Feature Extraction.** The next step includes converting this measurement into the result we need. In our case, volume is generally shown in terms of percentage(0% - 100%) or decibels(-65dB - 0dB). In order to convert the measurement we got to the volume we first normalise the measurement and then multiply it with a difference of max. Volume and min. Volume as shown in the following formula.

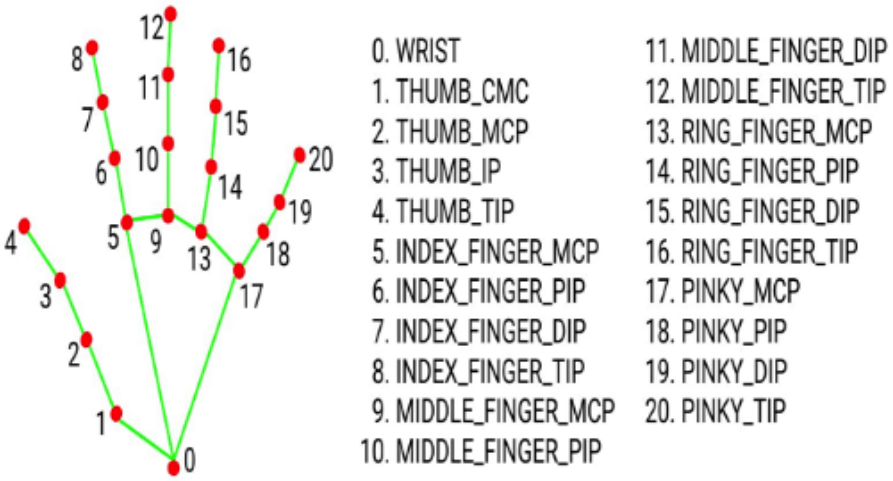
$$Volume = V_{min} + (d - d_{min}) \times \frac{V_{max} - V_{min}}{d_{max} - d_{min}}$$

We need to define dmin and dmax for it.

Now, we use ANN-***Artificial Neural Network*** such as convolutional Neural Network, to train a model on millions of images of the hand to recognise 21 key-points on our hand as shown in the figure(**a**). Using these key-points, ANN predicts the distance between the two specific key-points you specify to measure as the distance source for your volume controller. The rest of its calculation is done in python using other libraries such as numpy.

Now, sometimes, the distance d, we require is noisy. In-order to deal with it, we use FFT(**fast fourier transform**) based low pass filter.
FFT provides various information about volume, or the data received. We can also implement a gesture to increase base, or pitch of the sound and use FFT, to change those parameters accordingly. Signal processing plays a very important role in implementing this idea as all the frames we receive from the external world need to be sampled and digitized for the system to perform its calculations and other operations. It is plays very important role in understanding the hidden information of data

we get from the real world by frequency domain analysis.



figure(**a**)
Src: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

---

Approach-

STEP 01: We will set up the environment in python notebook by installing all the necessary libraries.
Required libraries: NumPy, matlplotlib, openCV, mediaPipe, pygame
STEP 02: Initialise the model to identify the hand. This is done using pre-trained model from mediaPipe.
STEP 03: Create a pathway to load frames from real world using openCV for real time processing.
STEP 04: Extract spatial coordinates. Note that these values are normalised between 0 and 1 by default.
STEP 05: Calculate pinch distance (d).
STEP 06: Apply normalisation/ linear interpolation to to the data you get since volume is going to lie between 0-100.
STEP 07: Audio processing to change bass and pitch (optional).
STEP 08: Implement UI to show key-points, volume level, bass value, pitch value, etc.
STEP 09: Implement break point to break from the loop of accessing camera whenever necessary.

PSEUDO_CODE-

GESTURE CONTROL VOLUME

Phase 01: Importing libraries and packages.

```
# install mediaPipe
!pip install mediapipe opencv-python
!pip uninstall mediapipe -y
!pip install mediapipe --user
```

Dealing with bass, pitch and real time audio playback system

```
!pip install scipy sounddevice

Defaulting to user installation because normal site-packages is not
writeable
Requirement already satisfied: scipy in
/home/dr_fhinx_nerd/.local/lib/python3.10/site-packages (1.15.3)
Requirement already satisfied: sounddevice in
/home/dr_fhinx_nerd/.local/lib/python3.10/site-packages (0.5.5)
Requirement already satisfied: numpy<2.5,>=1.23.5 in
/home/dr_fhinx_nerd/.local/lib/python3.10/site-packages (from scipy)
(2.2.6)
Requirement already satisfied: cffi in
/home/dr_fhinx_nerd/.local/lib/python3.10/site-packages (from
sounddevice) (2.0.0)
Requirement already satisfied: pycparser in
/home/dr_fhinx_nerd/.local/lib/python3.10/site-packages (from cffi-
>sounddevice) (3.0)
```

```
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
import cv2
import time
import numpy as np
import os
import matplotlib.pyplot as plt
```

Code to vary pitch and bass of the song and play song continuously while changing those parameters.

```
import sounddevice as sd
import numpy as np
from scipy.fft import fft, ifft
from scipy.io import wavfile
fs, data = wavfile.read('song.wav')
data = data.astype(np.float32) / 32768.0
if len(data.shape) > 1: data = data[:, 0]
current_audio_frame = 0
bass = 1
pitch = 0

def audio_callback(outdata, frames:1024,time, status):
    global current_audio_frame, data, bass, pitch

    start = current_audio_frame
    end = start + frames
    if end > len(data):
        chunk = data[start:]
        chunk = np.pad(chunk, (0, frames - len(chunk)))
        current_audio_frame = 0
    else:
        chunk = data[start:end]
        current_audio_frame = end
    spectrum = fft(chunk)
    spectrum[:15] *= float(bass)
    spectrum = np.roll(spectrum, pitch)
    chunk_out = np.real(ifft(spectrum))
    outdata[:] = chunk_out[:frames].reshape(-1, 1)
```

```python
# initialise camera
cap = cv2.VideoCapture(0, cv2.CAP_V4L2) # V4L2 is best for Zorin Linux
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
BaseOptions = mp.tasks.BaseOptions # tells the system where pre-trained model is located in the system.
HandLandmarker = mp.tasks.vision.HandLandmarker # this is the one that looks for hands.
HandLandmarkerOptions = mp.tasks.vision.HandLandmarkerOptions # Thsi holds all the required settings or modifications or things we require.
HandLandmarkerResult = mp.tasks.vision.HandLandmarkerResult # defines the structure of result.
VisionRunningMode = mp.tasks.vision.RunningMode # This defines the workFlow mode( input is image, video or live stream ).
```

```python
import sounddevice as sd
import numpy as np
from scipy.fft import fft, ifft
from scipy.io import wavfile
fs, data = wavfile.read('song.wav')
data = data.astype(np.float32) / 32768.0
if len(data.shape) > 1: data = data[:, 0]
current_audio_frame = 0
bass = 1
pitch = 0
def audio_callback(outdata, frames:1024,time, status):
    global current_audio_frame, data, bass, pitch

    # A. Get the next chunk of frames
    start = current_audio_frame
    end = start + frames
    if end > len(data):
        # Loop the song and pad if short
        chunk = data[start:]
        chunk = np.pad(chunk, (0, frames - len(chunk)))
        current_audio_frame = 0
    else:
        chunk = data[start:end]
        current_audio_frame = end
    # B. FFT Processing
    spectrum = fft(chunk)
    # 1. BASS BOOST (First 15 frequency bins)
    # Pinch (0.02) = 4.0x Bass, Open (0.15) = 1.0x Bass
    spectrum[:15] *= float(bass)

    # 2. PITCH SHIFT (Bin Shifting)
    # Pinch = Deep (-8 bins), Open = High (+15 bins)
    spectrum = np.roll(spectrum, pitch)

    # C. Reconstruct and Output
    # Ensure the shape is exactly what outdata expects (frames, 1)
    chunk_out = np.real(ifft(spectrum))
    outdata[:] = chunk_out[:frames].reshape(-1, 1)

# initialise camera
cap = cv2.VideoCapture(0, cv2.CAP_V4L2) # V4L2 is best for Zorin Linux
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

```
BaseOptions = mp.tasks.BaseOptions # tells the system where pre-
trained model is located in the system.
HandLandmarker = mp.tasks.vision.HandLandmarker # this is the one that
looks for hands.
HandLandmarkerOptions = mp.tasks.vision.HandLandmarkerOptions # Thsi
holds all the required settings or modifications or things we require.

HandLandmarkerResult = mp.tasks.vision.HandLandmarkerResult # defines
the structure of result.
VisionRunningMode = mp.tasks.vision.RunningMode # This defines the
workFlow mode( input is image, video or live stream ).


current_frame = None
last_vol_time = 0
# Create a hand landmarker instance with the live stream mode:
def print_result(result: HandLandmarkerResult, output_image: mp.Image,
timestamp_ms: int): # result stores data from HandLandmarkerResult;
output_image stores the live frame; timestamp_ms is the time exactly
when the frame has occured and its the value that we need to give
after rendering the image.
    # print('hand landmarker result: {}'.format(result))
    # mathematics required for volume control come here
    # detect landmarks
    global current_frame
    global last_vol_time
    global current_audio_frame
    global data, bass, pitch
    # Convert MediaPipe image back to BGR for OpenCV
    frame = cv2.cvtColor(output_image.numpy_view(), cv2.COLOR_RGB2BGR)
# converting back to original frame as that is what going to be
displayed using openCV.
    h, w, _ = frame.shape # height and width of frame.
    if result.hand_landmarks:
        for landmarks in result.hand_landmarks:
            # generaating skeleton

            cv2.line(frame, (int(landmarks[0].x * w),
int(landmarks[0].y * h)),(int(landmarks[1].x * w), int(landmarks[1].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[1].x * w),
int(landmarks[1].y * h)),(int(landmarks[2].x * w), int(landmarks[2].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[2].x * w),
int(landmarks[2].y * h)),(int(landmarks[3].x * w), int(landmarks[3].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[3].x * w),
int(landmarks[3].y * h)),(int(landmarks[4].x * w), int(landmarks[4].y
* h)),(150, 150, 150) , 3)
```

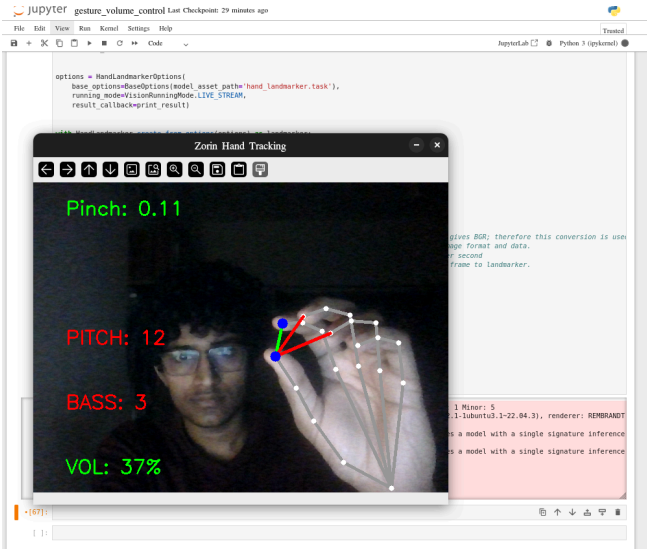
```
            cv2.line(frame, (int(landmarks[0].x * w),
int(landmarks[0].y * h)),(int(landmarks[5].x * w), int(landmarks[5].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[5].x * w),
int(landmarks[5].y * h)),(int(landmarks[6].x * w), int(landmarks[6].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[6].x * w),
int(landmarks[6].y * h)),(int(landmarks[7].x * w), int(landmarks[7].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[7].x * w),
int(landmarks[7].y * h)),(int(landmarks[8].x * w), int(landmarks[8].y
* h)),(150, 150, 150) , 3)


            cv2.line(frame, (int(landmarks[0].x * w),
int(landmarks[0].y * h)),(int(landmarks[9].x * w), int(landmarks[9].y
* h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[9].x * w),
int(landmarks[9].y * h)),(int(landmarks[10].x * w),
int(landmarks[10].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[10].x * w),
int(landmarks[10].y * h)),(int(landmarks[11].x * w),
int(landmarks[11].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[11].x * w),
int(landmarks[11].y * h)),(int(landmarks[12].x * w),
int(landmarks[12].y * h)),(150, 150, 150) , 3)

            cv2.line(frame, (int(landmarks[0].x * w),
int(landmarks[0].y * h)),(int(landmarks[13].x * w),
int(landmarks[13].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[13].x * w),
int(landmarks[13].y * h)),(int(landmarks[14].x * w),
int(landmarks[14].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[14].x * w),
int(landmarks[14].y * h)),(int(landmarks[15].x * w),
int(landmarks[15].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[15].x * w),
int(landmarks[15].y * h)),(int(landmarks[16].x * w),
int(landmarks[16].y * h)),(150, 150, 150) , 3)

            cv2.line(frame, (int(landmarks[0].x * w),
int(landmarks[0].y * h)),(int(landmarks[17].x * w),
int(landmarks[17].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[17].x * w),
int(landmarks[17].y * h)),(int(landmarks[18].x * w),
int(landmarks[18].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[18].x * w),
int(landmarks[18].y * h)),(int(landmarks[19].x * w),
int(landmarks[19].y * h)),(150, 150, 150) , 3)
            cv2.line(frame, (int(landmarks[19].x * w),
```

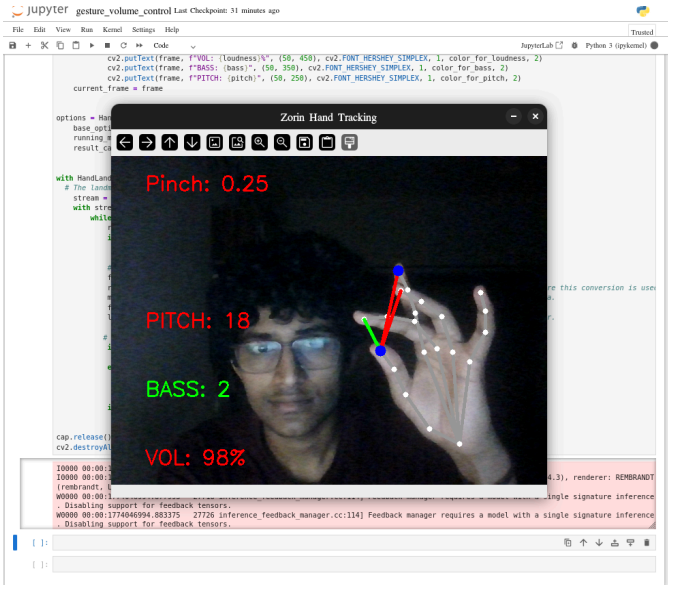
```
int(landmarks[19].y * h)),(int(landmarks[20].x * w),
int(landmarks[20].y * h)),(150, 150, 150) , 3)

            for lm in landmarks:
                cv2.circle(frame, (int(lm.x * w), int(lm.y * h)), 4,
(255, 255, 255), -1)

            thumb = landmarks[4]
            index = landmarks[8]
            mid_tip = landmarks[12]
            ring_tip = landmarks[16]

            distance_for_loudness = np.sqrt((index.x - thumb.x)**2 +
(index.y - thumb.y)**2)
            distance_for_bass = np.sqrt((mid_tip.x - thumb.x)**2 +
(mid_tip.y - thumb.y)**2)
            distance_for_pitch = np.sqrt((ring_tip.x - thumb.x)**2 +
(ring_tip.y - thumb.y)**2)

            # CONVERT TO PIXELS FOR DRAWING
            p1 = (int(thumb.x * w), int(thumb.y * h))
            p2 = (int(index.x * w), int(index.y * h))
            p3 = (int(mid_tip.x * w), int(mid_tip.y * h))
            p4 = (int(ring_tip.x * w), int(ring_tip.y * h))

            # Perform maths to change loudness of volume.
            loudness = int(np.interp(distance_for_loudness, [0.02,
0.25], [0, 100])) # linear interpolation
            bass = int(np.interp(distance_for_bass, [0.02, 0.2], [1,
5]))
            pitch = int(np.interp(distance_for_pitch, [0.02, 0.2], [-
10, 20]))
            if time.time() - last_vol_time > 0.1:
                os.system(f"amixer -D pulse sset Master {loudness}%
> /dev/null")
                last_vol_time = time.time()


            # VISUAL FEEDBACK
            # Line turns Green if pinched, Red if open
            color_for_loudness = (0, 255, 0) if distance_for_loudness
< 0.2 else (0, 0, 255)
            color_for_bass = (0, 255, 0) if distance_for_bass < 0.12
else (0, 0, 255)
            color_for_pitch = (0, 255, 0) if distance_for_pitch < 0.12
else (0, 0, 255)

            cv2.line(frame, p1, p2, color_for_loudness, 3)
            cv2.circle(frame, p1, 8, (255, 0, 0), cv2.FILLED)
            cv2.circle(frame, p2, 8, (255, 0, 0), cv2.FILLED)
```

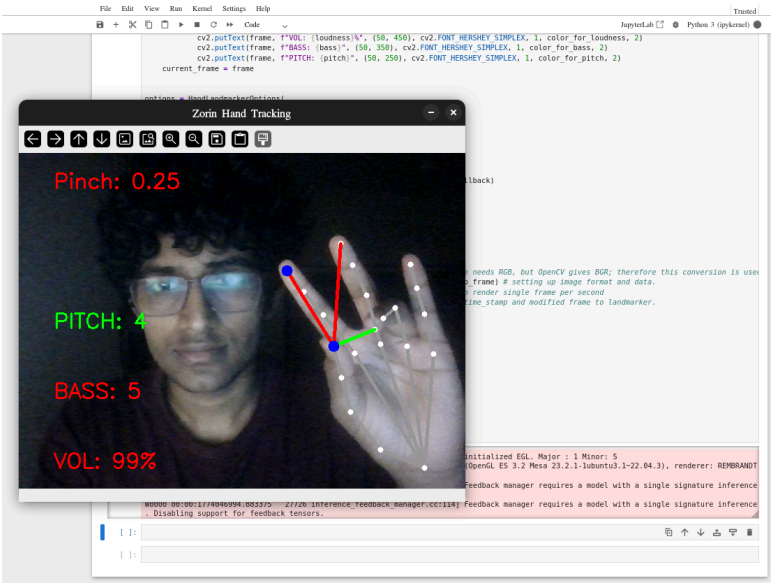
```
            cv2.line(frame, p1, p3, color_for_bass, 3)
            cv2.circle(frame, p1, 8, (255, 0, 0), cv2.FILLED)
            cv2.circle(frame, p2, 8, (255, 0, 0), cv2.FILLED)

            cv2.line(frame, p1, p4, color_for_pitch, 3)
            cv2.circle(frame, p1, 8, (255, 0, 0), cv2.FILLED)
            cv2.circle(frame, p2, 8, (255, 0, 0), cv2.FILLED)

            # Display distance; loudness
            cv2.putText(frame, f"Pinch: {round(distance_for_loudness,
2)}", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, color_for_loudness, 2)
            cv2.putText(frame, f"VOL: {loudness}%", (50, 450),
cv2.FONT_HERSHEY_SIMPLEX, 1, color_for_loudness, 2)
            cv2.putText(frame, f"BASS: {bass}", (50, 350),
cv2.FONT_HERSHEY_SIMPLEX, 1, color_for_bass, 2)
            cv2.putText(frame, f"PITCH: {pitch}", (50, 250),
cv2.FONT_HERSHEY_SIMPLEX, 1, color_for_pitch, 2)

    current_frame = frame


options = HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path='hand_landmarker.task'),
    running_mode=VisionRunningMode.LIVE_STREAM,
    result_callback=print_result)


with HandLandmarker.create_from_options(options) as landmarker:
  # The landmarker is initialized. Use it here.
    stream = sd.OutputStream(samplerate=fs, channels=1,
callback=audio_callback)
    with stream:
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                print("Ignoring empty camera frame.")
                continue
            # image processing to send frame to AI
            frame = cv2.flip(frame, 1)
            rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) #
MediaPipe needs RGB, but OpenCV gives BGR; therefore this conversion
is used.
            mp_image = mp.Image(image_format=mp.ImageFormat.SRGB,
data=rgb_frame) # setting up image format and data.
            frame_timestamp_ms = int(time.time() * 1000) # setting
time to render single frame per second
            landmarker.detect_async(mp_image, frame_timestamp_ms) #
send time_stamp and modified frame to landmarker.
```

```
        # displayinf the frame
        if current_frame is not None:
            cv2.imshow('Zorin Hand Tracking', current_frame)
        else:
            # If AI hasn't processed yet, just show the raw frame
            cv2.imshow('Zorin Hand Tracking', frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()
```

RESULTS-







OBSERVATION AND CONCLUSION:

The gesture-based volume control system is successfully designed and implemented using computer vision techniques. The system is able to detect hand gestures in real time through a webcam and control the system volume based on the distance between the thumb and index finger. When the distance between the fingers increases, the volume increases, and when the distance decreases, the volume reduces. This creates a smooth and intuitive way of controlling volume without using physical devices.

The hand tracking model is able to identify key points of the hand with good accuracy. Using these key points, the distance between selected fingers is calculated and then converted into a volume range between 0% and 100% using normalization. The system showed quick response with minimal delay, making it suitable for real-time applications. To improve performance, filtering techniques are applied to reduce noise and sudden fluctuations, resulting in smoother volume changes.

During testing, several observations were made. The system performed best under proper lighting conditions where the hand was clearly visible to the camera. In low lighting or cluttered backgrounds, the accuracy of detection was reduced, which sometimes affected the volume control. It was also observed that small unintentional hand movements caused slight variations in volume, although this effect was minimized using filtering.

Another important observation is that the system required the hand to remain within the camera frame. If the hand moved out of view or moved too fast, the detection temporarily failed. In some cases, when fingers overlapped or were not clearly visible, the system struggled to accurately identify the landmarks. Despite these limitations, the system is easy to use and the gesture felt natural for users.

In conclusion, this project demonstrates an effective application of gesture recognition for human-computer interaction. It provides a simple, contactless, and user-friendly method to control system volume. The integration of hand tracking, feature extraction, and signal processing helped achieve accurate and smooth performance.

Although the system works well, it still depends on environmental conditions like lighting and background. Future improvements can include using more advanced models, better cameras, or depth sensors to increase accuracy and robustness. Additional features like controlling brightness or media playback can also be added.

Overall, the project successfully shows how hand gestures can be used to interact with technology in a more natural and efficient way, reducing the need for traditional input devices.

LIMITATIONS:

The gesture-based volume control system has some limitations. It depends heavily on proper lighting conditions, and its performance reduces in low light or highly cluttered backgrounds. The accuracy of hand detection decreases when the hand moves too fast or goes out of the camera frame. Sometimes, small unwanted hand movements can cause fluctuations in volume. The system also struggles when fingers overlap or are not clearly visible. Additionally, it requires a continuous camera feed, which may consume system resources. These factors make the system less reliable in uncontrolled environments and limit its performance in real-world applications.

Video demonstration:
https://drive.google.com/file/d/16IQjZlTa0QoHIeuBpR2PMWQnx5bpuTLS/view?usp=sharing

Github link:
https://github.com/manikantabangarshettar06-stack/GestureVolumeControl

REFERENCES:

- **OpenCV Documentation: https://opencv.org/**
-
- **NumPy Documentation: https://numpy.org/doc/**
-
- **Gonzalez, R. C., & Woods, R. E. (2008). Digital Image Processing. Pearson Education.**
-
- **Oppenheim, A. V., & Schafer, R. W. (2010). Signals and Systems. Prentice Hall.**
-
- **Proakis, J. G., & Manolakis, D. G. (2006). Digital Signal Processing: Principles, Algorithms, and Applications.**
-
- **Python Official Documentation: https://docs.python.org/3/**